

Exercise Sheet 3

Geometric Transformations

Question 1.

In the lecture, you learned about rotation matrices. Please complete the following tasks in this context.

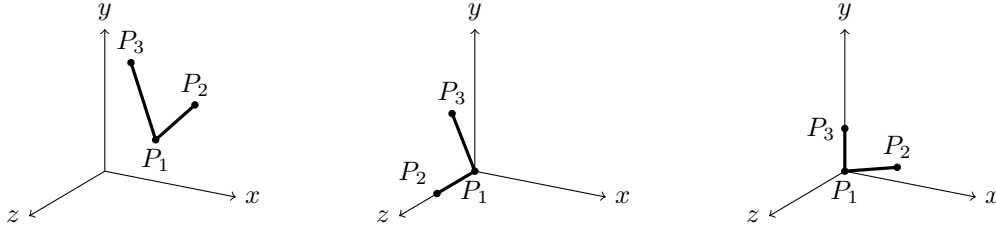


Figure 1: (left) initial position, (middle) final state for part (a), and (right) final state for part (b).

- (a) (3 points) Demonstrate that for arbitrarily directed line segments P_1P_2 and P_1P_3 (see Figure 1), the orthogonal rotation matrix

$$R = \begin{pmatrix} r_{1x} & r_{2x} & r_{3x} \\ r_{1y} & r_{2y} & r_{3y} \\ r_{1z} & r_{2z} & r_{3z} \end{pmatrix},$$

with

$$\begin{aligned} R_z &= (r_{1z}, r_{2z}, r_{3z}) = \frac{P_1P_2}{|P_1P_2|}, \\ R_x &= (r_{1x}, r_{2x}, r_{3x}) = \frac{P_1P_3 \times P_1P_2}{|P_1P_3 \times P_1P_2|}, \text{ and} \\ R_y &= (r_{1y}, r_{2y}, r_{3y}) = R_z \times R_x, \end{aligned}$$

implements a rotation such that after the transformation P_1P_2 is parallel to the z-axis and P_1P_3 is parallel to the yz-plane (see Figure 1 (middle)).

- (b) (3 points) Provide a sequence of transformations that adjusts the directed line segments P_1P_2 and P_1P_3 in Figure 1 (left) so that point P_1 is at the coordinate origin and point P_3 lies on the positive y-axis. Point P_2 should be in the xy-plane after the transformation (see Figure 1 (right)). Provide the derived matrix for each transformation step, along with a simple sketch illustrating each step. The matrices do not need to be expanded!
- (c) (2 points) Provide the rigid-body transformation for the transition of the line segments as described in part b.

Question 2.

With this exercise sheet, a framework is available for download in which the programming tasks should be solved. The framework creates several textured spheres that need to be moved and rotated to the correct positions to form an animated (simplified) solar system. A solar system consists of a central body (Sun) and several local planetary systems (Earth-Moon system).

Each astronomical object (**astro_object**, representing stars, planets, and moons) in this system is defined by various parameters:

- **orbitDistance**: The distance to the center of the (local) system.

- **orbitPeriod**: The duration of one orbit (around the (local) center).
- **size**: The radius of the object at the equator (x -coordinate) and at the pole (y -coordinate).
- **rotAxis**: The inclination of the rotation axis in degrees.
- **rotPeriod**: The duration of one rotation.

All astronomical objects (except the Sun) are organized into planetary systems. A planetary system (**planet_system**) is defined by:

- **objects**: List of astronomical objects in this planetary system.
- **orbitPlane**: The plane on which the center of the planetary system orbits the Sun.
- **orbitDistance**: Distance of the center to the Sun.
- **orbitPeriod**: Duration of one rotation around the Sun.
- **localSystemPlane**: The local plane on which the objects orbit the center.

Rotation/orbit durations are defined in days, and axis inclinations in the x direction. Use existing functions from 'GLM' (especially **glm::rotate**, **glm::translate**, and **glm::scale**) for all transformations.

- (a) (6 points) Edit the **main** function in the **main.cpp** file and save the correct transformation matrices to describe the positions of the solar system in **worldMatrices** (order: Sun, Earth, Moon, Pluto, Charon). Use the values from the variables **sun** and **planetSystems**. The hierarchy and order of the objects in **planetSystems** are as follows:
- Earth-Moon System: (a) Earth (b) Moon
 - Pluto-Charon System: (a) Pluto (b) Charon
- (b) (2 points) Change the calculation of the matrix **vp** (View-Projection Matrix) so that the Earth-Moon system (!) serves as the reference system for the camera.
- (c) (4 points) Animate the created solar system. Use the function **glframework::getTime()** to make self-rotation and orbits variable over time. The function returns the current time in seconds. One day in the animation should last five seconds.

All areas where you need to change source code are marked with a comment of the form `//TODO:`.

Submit the written tasks, the modified framework, and the result images by **Monday, June 2, 2026, 11:55PM** via the Moodle platform in the form of a ZIP file.